

3. 함수 & 파이썬 입출력

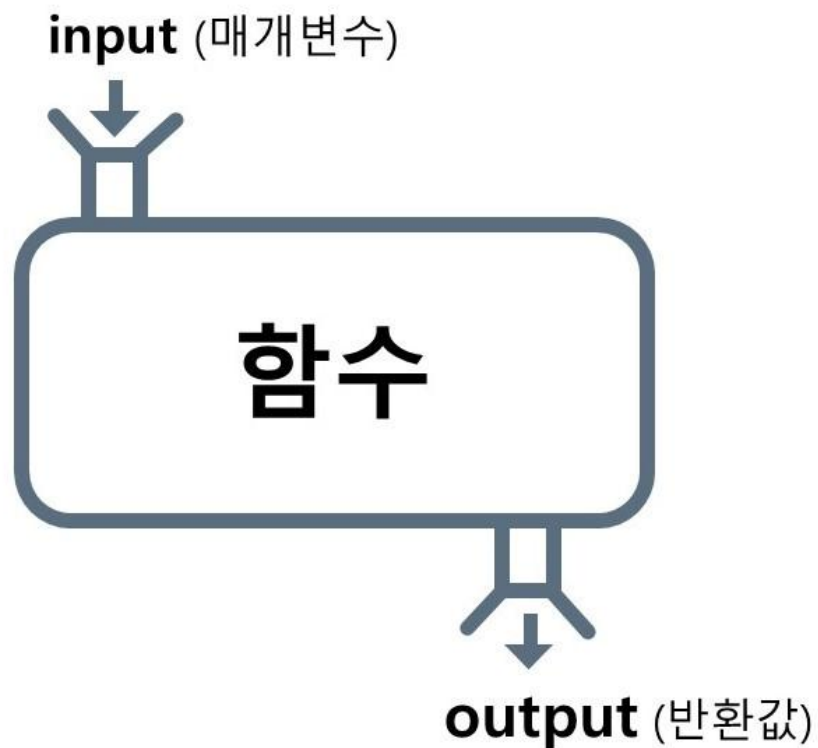
3. 함수 & 파이썬 입출력



- 3-1 함수
- 3-2 사용자 입출력
- 3-3 파일 읽기&쓰기
- 3-4 프로그램 입출력

3-1. 함수

3-1. 함수



- 입력값을 가지고,
- 어떤 일을 수행한 후,
- 그 결과물을 내어 놓는 것



■ 왜 함수를 사용해야 할까?

- 반복 작업의 효율화: 반복적으로 사용되는 로직을 함수로 묶어서 사용
- 가독성-유지보수 향상 : 기능 단위로 분리되어 흐름 파악과 오류 추적이 쉬움



■ 파이썬 함수의 구조

```
def 함수_이름(매개변수):  
    수행할_문장1  
    수행할_문장2  
    ...  
    return 반환값
```

■ 함수 예시

```
# 함수 정의  
def add(a, b):  
    result = a + b  
    return result
```

```
# 함수 호출  
>>> a = add(3, 4)  
>>> print(a)
```

7

■ 1-1) 입력값이 없는 함수

```
def say():  
    return 'hello, world!'
```

입력값이 없음 = 매개변수가 없음

* 입력값은 X, 반환값은 O

```
>>> a = say()  
>>> print(a)  
hello, world!
```

■ 1-2) 반환값이 없는 함수

```
def add(a,b):  
    print("%d, %d의 합은 %d입니다." % (a, b, a+b))
```

```
>>> add(3,4)  
3, 4의 합은 7입니다.
```

■ 1-3) 입력값도, 반환값도 없는 함수

```
def say():  
    print('Hi')
```

* 입력 인수를 받는 매개변수도 없고 return 문도 없으니 입력값도, 반환값도 없는 함수.

```
>>> say()  
Hi!
```



■ 1-4) 매개변수를 지정하여 호출하기

```
def sub(a, b):  
    return a - b
```

* 함수를 호출할 때 매개변수를 지정할 수 있음.

```
>>> result = sub(b=5, a=3) # b에 5, a에 3 전달  
>>> print(result)  
-2
```

- 1-5) 가변 매개변수, args 매개 변수가 몇 개인지 모를 때

```
def 함수_이름(*매개변수): # 괄호 안의 매개변수 부분이 -> *매개변수 로 바뀜
    수행할 문장
    ...
```

```
def add_many(*args):
    result = 0
    for i in args:
        result = result + i # *args에 입력받은 모든 값을 더한다.
    return result
```

■ 1-5) 가변 매개변수, args

```
>>> result = add_many(1,2,3)
```

```
>>> print(result)
```

```
6
```

```
>>> result = add_many(1,2,3,4,5,6,7,8,9,10)
```

```
>>> print(result)
```

```
55
```

■ 1-5) 가변 매개변수, args

- 여러 개의 입력을 처리할 때, 매개변수로 *args 하나만 사용할 수 있는 것은 아님.

```
def add_mul(choice, *args):  
    if choice == "add":  
        result = 0  
        for i in args:  
            result = result + i  
    elif choice == "mul":  
        result = 1  
        for i in args:  
            result = result * i  
  
    return result
```

■ 1-5) 가변 매개변수, args

```
>>> result = add_mul('add', 1,2,3,4,5)
```

```
>>> print(result)
```

```
15
```

```
>>> result = add_mul('mul', 1,2,3,4,5)
```

```
>>> print(result)
```

```
120
```

■ 1-6) 키워드 매개변수, kwargs

- 키워드 매개변수는 함수 호출 시 키워드=값 형태로 전달하는 매개변수를 받을 때 사용함.
- 매개변수 앞에 별 2개(**)를 붙임.

```
def 함수_이름(**매개변수): # 괄호 안의 매개변수 부분이 -> **매개변수
    수행할 문장
    ...
```

```
def print_kwargs(**kwargs):
    print(kwargs)
```

```
>>> print_kwargs(a=1)
{'a' : 1}
```

■ 1-6) 키워드 매개변수, kwargs

```
def create_profile(**info):  
    print("=== 프로필 정보 ===")  
    for key, value in info.items():  
        print(f"{key}: {value}")
```

```
>>> create_profile(name="홍길동", age=30, city="서울")  
===프로필 정보===  
name : 홍길동  
age : 30  
city : 서울
```

■ 1-7) 일반 매개변수, 가변 매개변수, 키워드 매개변수 함께 사용하기

```
def mixed_function(name, *args, **kwargs):  
    print(f"이름: {name}")  
    print(f"추가 인수들: {args}")  
    print(f"키워드 인수들: {kwargs}")
```

```
>>> mixed_function('홍길동', 1, 2, 3, age=25, city='서울')
```

```
이름: 홍길동  
추가 인수들: (1, 2, 3)  
키워드 인수들: {'age' : 25, 'city': '서울'}
```

■ 1-8) 함수의 반환값은 항상 하나

```
def add_and_mul(a,b):  
    return a+b, a*b
```

```
>>> result = add_and_mul(3,4)  
>>> print(result)  
(7, 12)
```

반환값을 받아들인 변수는 'result' 하나. 하지만 오류는 발생하지 않음. 왜?

함수의 반환값은 언제나 1개.

따라서, add_and_mul 함수의 반환 a+b, a*b는 튜플값 하나인 (a+b, a*b)로 반환됨.

■ 1-9) return 의 특수 용도: 함수 강제 종료

함수를 빠져나가고 싶을 때, 제어문의 break 처럼 return을 사용해서 함수를 즉시 빠져나갈 수 있음.

```
def say_nick(nick):  
    if nick == "바보":  
        return  
    print(f"나의 별명은 {nick} 입니다.")
```

```
>>> say_nick('야호')  
나의 별명은 야호입니다.  
>>> say_nick('바보') # return을 만나 함수가 종료됨  
>>>
```

■ 1-10) 매개변수에 초기값 미리 설정하기

```
def say_myself(name, age, woman=True):    # man 매개변수에 미리 값을 지정
    print( "이름: %s", %name)
    print( "나이: %d 살", %age)
    if woman:
        print("성별: 여")
    else:
        print("성별: 남")
```

(주의)
초기값을 미리 지정하고 싶은 매개변수는
항상 뒤쪽에 놓아야 함!

```
>>> say_myself("김나나", 27)
```

```
>>> say_myself("김나나", 27, False)
```

```
say_myself("김나나", 27)
✓ 0.0s
이름: 김나나
나이: 27 살
성별: 여

say_myself("김나나", 27, False)
✓ 0.0s
이름: 김나나
나이: 27 살
성별: 남
```

■ 1-11) 함수 안에서 선언한 변수의 효력 범위

함수 안에서 사용하는 매개변수는 함수 밖의 변수 이름과 전혀 상관이 없음

```
a = 1
def vartest(a):
    a += 1
```

```
vartest(a)
print(a)
>>> 1
```

```
print(vartest(a))
>>> None
```

■ 1-12) 함수 안에서 함수 밖의 변수를 변경하는 방법

1. return 사용하기

```
a = 1
def vartest(a):
    a = a + 1
    return a
```

```
a = vartest(a)
print(a)
```

```
>>> 2
```

2. global 명령어 사용하기

```
a = 1
def vartest():
    global a
    a = a + 1
```

```
vartest()
print(a)
```

```
>>> 2
```

global 명령어란?

함수 안에서 함수 밖에서 선언된 변수를 수정할 때 사용하는 키워드

함수는 독립적으로 존재하는 것이 좋음.
따라서, global 명령어는 되도록 사용하지 않는 것이 좋다!

■ 1-13) lambda 예약어

def와 동일한 역할을 함. 보통 함수를 한 줄로 간결하게 만들 때 사용.

함수명 = **lambda** 매개변수 1, 매개변수 2, ... : 매개변수를_이용한_표현식

```
add = lambda a, b: a + b
result = add(3, 4)
print(result)
>>> 7
```

```
say = lambda a, b: f"안녕! 나는 {a}야, 나는 {b}살이야."
result = say("왈왈이", 10)
print(result)
>>> 안녕! 나는 왈왈이야, 나는 10살이야.
```

3-1. 함수



2분

- 예제 1) "안녕 친구들! 잘 부탁해!" 를 출력하는 함수(hello)를 작성하시오.

* 실행 예시

```
print(hello())
```

[5] ✓ 0.0s

... 안녕 친구들! 잘 부탁해!

```
hello()
```

✓ 0.0s

'안녕 친구들! 잘 부탁해!'

3-1. 함수



2분

- 예제 1) "안녕 친구들! 잘 부탁해!" 를 출력하는 함수(hello)를 작성하시오. (return 사용할 것)

```
def hello():  
    return '안녕 친구들! 잘 부탁해'
```

* 실행 예시

```
[5] print(hello())  
✓ 0.0s  
... 안녕 친구들! 잘 부탁해!
```

```
hello()  
✓ 0.0s  
'안녕 친구들! 잘 부탁해!'
```

3-1. 함수



2분

- 예제 2) 이름과 나이를 입력받아 화면에 출력 하는 함수(print_info)를 완성하시오.

A large, empty rectangular box with a black border, intended for the user to write the Python function code.

* 실행 예시

```
print_info("하하하", 17)
```

✓ 0.0s

```
===== 정보 출력 =====
```

```
이름: 하하하
```

```
나이: 17
```

```
출력이 완료되었습니다.
```

3-1. 함수



2분

- 예제 2) 이름과 나이를 입력받아 화면에 출력 하는 함수(print_info)를 완성하시오.

```
def print_info(name, age):  
    print("==== 정보 출력 =====")  
    print(f"이름: {name}")  
    print(f"나이: {age}")  
    print("출력이 완료되었습니다.")
```

* 실행 예시

```
print_info("하하하", 17)  
✓ 0.0s  
  
==== 정보 출력 =====  
이름: 하하하  
나이: 17  
출력이 완료되었습니다.
```

3-1. 함수



- 예제 3) 세 개의 매개변수를 받고, for문을 사용하여 최대-최소 차이를 구하는 함수를 작성하시오.
- (min, max 함수 사용 금지)

5분

```
def diff_max_min(a, b, c):  
    nums = [a, b, c]  
  
    largest = nums[0]  
    smallest = nums[0]
```

* 실행 예시

```
diff_max_min(3,10,5)  
✓ 0.0s  
7
```

3-1. 함수

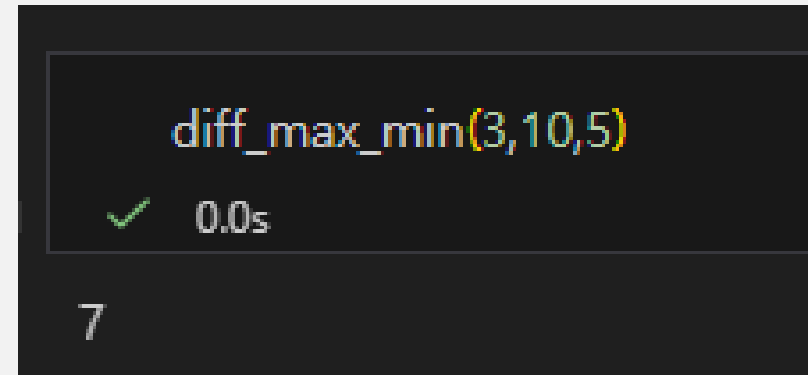


- 예제 3) 세 개의 매개변수를 받고, for문을 사용하여 최대-최소 차이를 구하는 함수를 작성하시오.
- (min, max 함수 사용 금지)

5분

```
def diff_max_min(a, b, c):  
    nums = [a, b, c]  
  
    largest = nums[0]  
    smallest = nums[0]  
  
    for n in nums:  
        if n > largest:  
            largest = n  
        if n < smallest:  
            smallest = n  
  
    return largest - smallest
```

* 실행 예시

A screenshot of a terminal window with a dark background. The first line shows the function call 'diff_max_min(3,10,5)' in a light blue font. The second line shows a green checkmark followed by '0.0s' in a light blue font. The third line shows the result '7' in a light blue font.

```
diff_max_min(3,10,5)  
✓ 0.0s  
7
```

3-1. 함수



2분

- 예제 4) 입력 개수가 정해지지 않은 숫자들을 모두 출력하시오.



* 실행 예시

```
print_numbers(1, 3, 5)  
print_numbers(10, 20, 30, 40)
```

✓ 0.0s

입력된 숫자들:

1

3

5

입력된 숫자들:

10

20

30

40

- 예제 4) 입력 개수가 정해지지 않은 숫자들을 모두 출력하시오.

```
def print_numbers(*numbers):  
    print("입력된 숫자들:")  
    for num in numbers:  
        print(num)
```

* 실행 예시

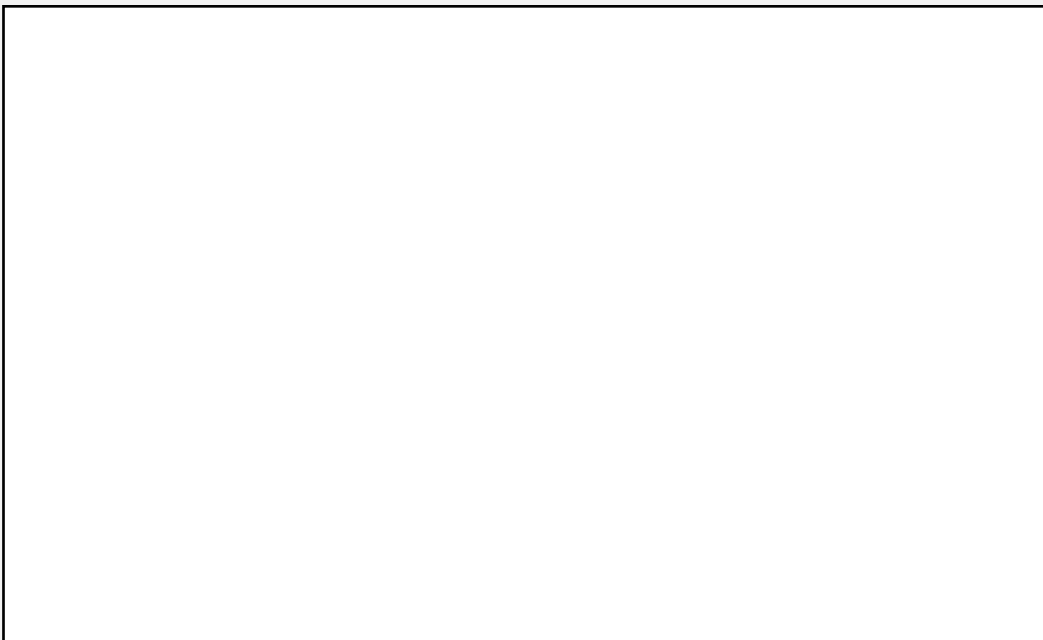
```
print_numbers(1, 3, 5)  
print_numbers(10, 20, 30, 40)  
✓ 0.0s  
입력된 숫자들:  
1  
3  
5  
입력된 숫자들:  
10  
20  
30  
40
```

3-1. 함수



3분

- 예제 5) 캐릭터 정보를 키워드 매개변수로 받아 화면에 출력하는 함수를 완성하시오.



* 실행 예시

```
show_character(  
    name="마법사 밍",  
    job="Wizard",  
    level=10,  
    skill="Fireball"  
)
```

✓ 0.0s

```
=== 캐릭터 정보 ===  
name : 마법사 밍  
job : Wizard  
level : 10  
skill : Fireball
```

3분

- 예제 5) 캐릭터 정보를 키워드 매개변수로 받아 화면에 출력하는 함수를 완성하시오.

```
def show_character(**info):  
    print("=== 캐릭터 정보 ===")  
  
    for key, value in info.items():  
        print(f"{key} : {value}")
```

* 실행 예시

```
show_character(  
    name="마법사 밍",  
    job="Wizard",  
    level=10,  
    skill="Fireball"  
)
```

✓ 0.0s

```
=== 캐릭터 정보 ===  
name : 마법사 밍  
job : Wizard  
level : 10  
skill : Fireball
```

3-1. 함수



- 예제 6) 나이를 입력받아 영화관 입장 여부를 판단하는 프로그램을 만드시오.
- 조건 : 19세 이상이면 입장 가능, 19세 미만이면 입장 불가

3분

check_pass =

* 실행 예시

```
print(check_pass(20))
print(check_pass(15))
✓ 0.0s
PASS
STOP
```

3-1. 함수



- 예제 6) 나이를 입력받아 영화관 입장 여부를 판단하는 프로그램을 만드시오.
- 조건 : 19세 이상이면 입장 가능, 19세 미만이면 입장 불가

3분

```
check_pass = lambda age: "PASS" if age >= 19 else "STOP"
```

* 실행 예시

```
print(check_pass(20))  
print(check_pass(15))  
✓ 0.0s  
PASS  
STOP
```

3-2. 사용자 입출력

3-2. 사용자 입출력



- 언제 사용? 사용자가 입력한 값을 변수에 대입하고 싶을 때
- input 사용하기

```
>>> a = input()
포빅아 32기 파이팅

>>> a
'포빅아 32기 화이팅'
```

포빅아 32기 파이팅

Press 'Enter' to confirm your input or 'Escape' to cancel

- 프롬프트를 띄워 사용자 입력 받기

```
>>> number = input("숫자를 입력하세요: ")
숫자를 입력하세요: 32

>>> print(number)
32
```

3-2. 사용자 입출력



- Input으로 입력되는 모든 것은 문자열로 취급

```
>>> type(number)
str
```

- 입력값을 숫자로 변환하기

```
>>> age = input("나이를 입력하세요: ")
나이를 입력하세요: 25
```

```
>>> age = int(age) # 문자열을 정수로 변환
>>> print(age + 1)
26
```

```
>>> age = float(age) # 정수형을 실수형으로 변환
>>> print(age/100)
0.25
```

3-2. 사용자 입출력



- print 문의 용도는 데이터를 출력하는 것
- print 문 자세히 알기

큰따옴표로 둘러싸인 문자열은 + 연산과 동일함.

```
>>> print("life" "is" "too short")  
lifeistoo short
```

```
>>> print("life"+"is"+"too short")  
lifeistoo short
```

문자열 띄어쓰기는 쉼표로 함.

```
>>> print("life", "is", "too short")  
life is too short
```

sep 매개변수로 구분자 설정

```
>>> print("2026", "02", "23", sep="-")  
2026-02-23
```

■ 한 줄에 결과값 출력하기

```
>>> for i in range(10):  
    print(i, end=' ')
```

```
0 1 2 3 4 5 6 7 8 9
```

- 한 줄에 결과값을 이어서 출력하려면 end 매개변수를 사용해 끝 문자를 지정해야 함.
- end의 기본값은 줄바꿈(\n) 문자임

3-2. 사용자 입출력



10분

- 예제 7) 점수 기반 Top-K 함수 만들기
 - lambda 사용하기
 - 상위 k개만 반환
 - "Top K 값을 입력하세요: "라는 문구와 함께 Top K값을 입력받으시오.

```
predictions = [  
    ("cat", 0.2),  
    ("dog", 0.85),  
    ("bird", 0.6),  
    ("tiger", 0.91),  
]
```

* 실행 예시

2

Top K 값을 입력하세요: (Press 'Enter' to confirm or 'Escape' to cancel)

```
print(f"Top K : {k}") #k가 2일 때  
print(result)
```

✓ 0.0s

```
Top K : 2  
[('tiger', 0.91), ('dog', 0.85)]
```

3-2. 사용자 입출력



10분

- 예제 7) 점수 기반 Top-K 함수 만들기
 - lambda 사용하기
 - 상위 k개만 반환
 - "Top K 값을 입력하세요: "라는 문구와 함께 Top K값을 입력받으시오.

```
def get_top_k(predictions, k):  
    sorted_preds = sorted()  
    return sorted_preds[:k]
```

k =

result =

```
print(f"Top K : {k}") #k가 2일 때  
print(result)
```

✓ 0.0s

```
Top K : 2  
[('tiger', 0.91), ('dog', 0.85)]
```

10분

- 예제 7) 점수 기반 Top-K 함수 만들기
 - lambda 사용하기
 - 상위 k개만 반환
 - "Top K 값을 입력하세요: "라는 문구와 함께 Top K값을 입력받으시오.

```
def get_top_k(predictions, k):  
    sorted_preds = sorted(predictions, key=lambda x: x[1], reverse=True)  
    return sorted_preds[:k]
```

```
k = int(input("Top K 값을 입력하세요: "))  
result = get_top_k(predictions, k)
```

3-3. 파일

3-3. 파일 읽기&쓰기



■ 파일 생성하기

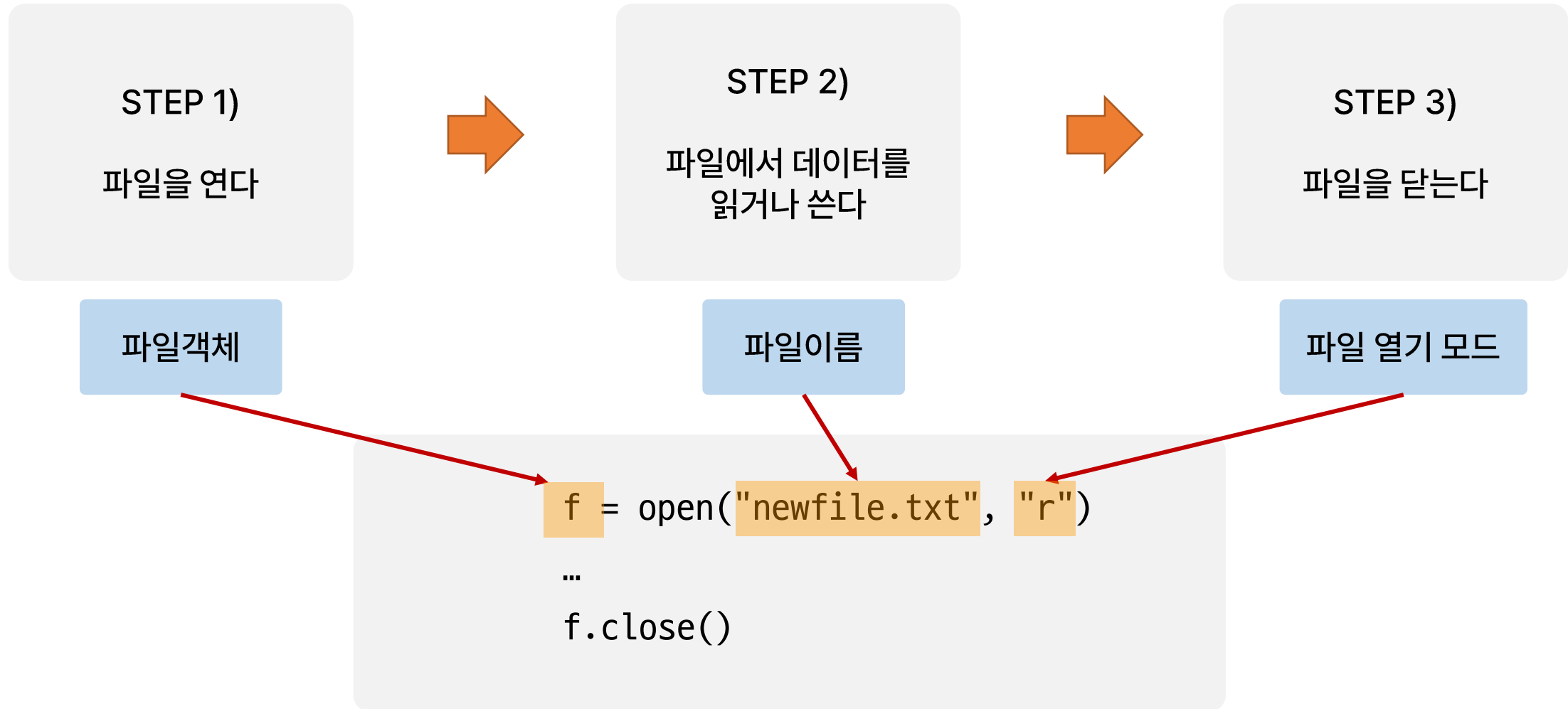
```
f = open("newfile.txt", 'w') #open(파일_이름, 파일_열기_모드)
f.close()
```

파일을 생성하기 위해 파이썬 내장 함수 open을 사용

파일열기모드	설명
r	읽기 모드: 파일을 읽기만 할 때 사용한다.
w	쓰기 모드: 파일에 내용을 쓸 때 사용한다. 파일의 처음부터 쓴다. 파일이 없으면 생성된다. 파일이 존재하면 기존의 내용은 지워진다.
a	추가 모드: 파일의 마지막에 새로운 내용을 추가할 때 사용한다.

- 만약 'newfile.txt' 파일을 다른 경로에 생성하고 싶다면? **practice 디렉토리를 만들어 그 안에 생성**
`f = open("./practice/newfile.txt", 'w')`

3-3. 파일 읽기&쓰기





■ 파일 쓰기

```
f = open("./practice/newfile.txt", 'w')
for i in range(1, 11):
    data = "%d번째 줄입니다.\n" % i
    f.write(data)
f.close()
```

3-3. 파일 읽기&쓰기



■ 파일 읽기

```
f = open("./practice/newfile.txt", 'r')
```

```
line = f.readline()
print(line)
f.close()
```

>>> 1번째 줄입니다.

1. readline 함수

파일에서 한 줄 읽어옴

2. readlines 함수

모든 라인을 한꺼번에 읽어서
각각의 줄을 요소로 갖는 리스트를 반환

```
f = open("C:/result/newfile.txt", 'r')
```

```
lines = f.readlines()
for line in lines:
    print(line)
f.close()
```

1번째 줄입니다.
2번째 줄입니다.
3번째 줄입니다.
4번째 줄입니다.
5번째 줄입니다.
6번째 줄입니다.
7번째 줄입니다.
8번째 줄입니다.
9번째 줄입니다.
10번째 줄입니다.

3-3. 파일 읽기&쓰기



■ 파일 읽기

3. read 함수

파일을 전부 읽은 문자열을 반환

```
f = open("./practice/newfile.txt", 'r')
data = f.read()
print(data)
f.close()
```

```
1번째 줄입니다.
2번째 줄입니다.
3번째 줄입니다.
4번째 줄입니다.
5번째 줄입니다.
6번째 줄입니다.
7번째 줄입니다.
8번째 줄입니다.
9번째 줄입니다.
10번째 줄입니다.
```

4. for문

파일 객체(f)는 for 문과 함께 사용하여 파일을 줄 단위로 읽을 수 있음.

```
f = open("./practice/newfile.txt", 'r')
for line in f:
    print(line)
f.close()
```

```
1번째 줄입니다.
2번째 줄입니다.
3번째 줄입니다.
4번째 줄입니다.
5번째 줄입니다.
6번째 줄입니다.
7번째 줄입니다.
8번째 줄입니다.
9번째 줄입니다.
10번째 줄입니다.
```

■ with 문 사용하기 = 파일을 열고 자동으로 닫기

```
f = open("foo.txt", 'w')  
f.write("Life is too short, you need python")  
f.close()
```

- 파일을 열면(open) 항상 닫아(close) 주어야 함. 파이썬 with 문이 이런 역할을 해 줌.

```
with open("foo.txt", 'w') as f:  
    f.write("Life is too short, you need python")
```

- with 문을 사용하면 with 블록(with 문에 속해 있는 문장)을 벗어나는 순간 열린 파일 객체 f가 자동으로 닫힘.

3-3. 파일 읽기 & 쓰기



- 예제 8) 파일에 내용을 저장하고 다시 읽어오기. (예제8.txt 파일명으로 저장하기)

10분

1단계 : 파일 쓰기(생성)

print("파일 저장 완료!")

2단계 : 파일 내용 읽기

print()

print("==== 파일 내용 ====")

print()

파일 저장 완료!

==== 파일 내용 ====
파이썬 파일 입출력 연습 중입니다.
파일에 글이 잘 저장되었나요?

3-3. 파일 읽기 & 쓰기



- 예제 8) 파일에 내용을 저장하고 다시 읽어오기. (예제8.txt 파일명으로 저장하기)

10분

```
# 1단계 : 파일 쓰기(생성)
with open("예제8.txt", "w", encoding="utf-8") as f:
    f.write("파이썬 파일 입출력 연습 중입니다.\n")
    f.write("파일에 글이 잘 저장되었나요?\n")
print("파일 저장 완료!")

# 2단계 : 파일 내용 읽기
print()
with open("예제8.txt", "r", encoding="utf-8") as f:
    content = f.read()
print("==== 파일 내용 =====")
print(content)
```

파일 저장 완료!

==== 파일 내용 =====
파이썬 파일 입출력 연습 중입니다.
파일에 글이 잘 저장되었나요?

3-4. 프로그램 입출력

3-4. 프로그램 입출력



■ sys 모듈

- 파이썬에서는 sys 모듈을 사용하여 프로그램에 인수를 전달할 수 있음.
- Sys 모듈의 argv는 프로그램 실행 시 전달된 인수를 담는 리스트.



```
(base) C:\Users\duqht\Downloads>python sys1.py aaa bbb ccc  
aaa  
bbb  
ccc
```

- 모듈을 사용하려면 `import sys` 처럼 `import` 명령어를 사용해야 함.

```
# sys1.py  
import sys  
args = sys.argv[1:]  
for i in args:  
    print(i)
```

3-4. 프로그램 입출력



■ sys 모듈

- 모듈을 사용하려면 import sys 처럼 import 명령어를 사용해야 함.

```
# sys2.py
import sys
args = sys.argv[1:]
for i in args:
    print(i.upper(), end=' ')
```

- 문자열 관련 함수인 upper()을 사용하여 전달된 인수를 모두 대문자로 바꾸어 주는 프로그램

```
>>> python sys2.py life is too short, you need python
LIFE IS TOO SHORT, YOU NEED PYTHON
```

```
(base) C:\Users\dught\Downloads>python sys2.py life is too short, you need python
LIFE IS TOO SHORT, YOU NEED PYTHON
```

3-4. 프로그램 입출력



- 예제 9) sys 모듈을 사용해 실행 인자를 처리하는 예제

5분



* 실행 예시

```
>>> python ex9.py 돌리 고길동 마이콜 도우너
```

```
===== 출석 체크 =====  
돌리님 출석 완료!  
고길동님 출석 완료!  
마이콜님 출석 완료!  
도우너님 출석 완료!
```

3-4. 프로그램 입출력



- 예제 9) sys 모듈을 사용해 실행 인자를 처리하는 예제

5분

```
import sys

names = sys.argv[1:]
print("==== 출석 체크 =====")
for name in names:
    print(f"{name} 출석 완료!")
```

* 실행 예시

```
>>> python ex9.py 돌리 고길동 마이콜 도우너
```

```
==== 출석 체크 =====
돌리님 출석 완료!
고길동님 출석 완료!
마이콜님 출석 완료!
도우너님 출석 완료!
```

Practice Assignment

■ 과제 1

- 몇 개의 재료가 들어갈지 모름. (매개변수명: ingredients)
- 만약 입력된 재료가 하나도 없다면, "주문할 재료를 선택해주세요." 라고 출력하고 즉시 함수를 종료하기

```
def order_snack():  
    # 1. 재료가 없는 경우 (강제 종료)
```

```
    # 2. 재료가 있는 경우 출력
```

```
order_snack()  
✓ 0.0s  
주문할 재료를 선택해주세요.  
  
order_snack("치즈", "햄", "토마토")  
✓ 0.0s  
주문하신 재료: ('치즈', '햄', '토마토')  
'주문이 완료되었습니다!'
```

■ 과제 2

- halo.txt 파일 내용을 한 줄씩 읽어 모두 대문자로 바꿔주는 프로그램을 작성하시오.
 - 파일명 입력 : input() 함수를 사용하여 읽고 싶은 파일 이름을 입력받기('halo.txt' 파일명 입력하기)
 - 파일 읽기 : with문과 readlines() 함수를 사용하여 파일 내용을 읽기
 - 대문자 변환 : upper() 함수를 사용하기

```
# halo.txt
```

```
hello  
don't give up  
you're doing great
```



```
test.txt
```

파일 이름을 입력하세요: (Press 'Enter' to confirm or 'Escape' to cancel)

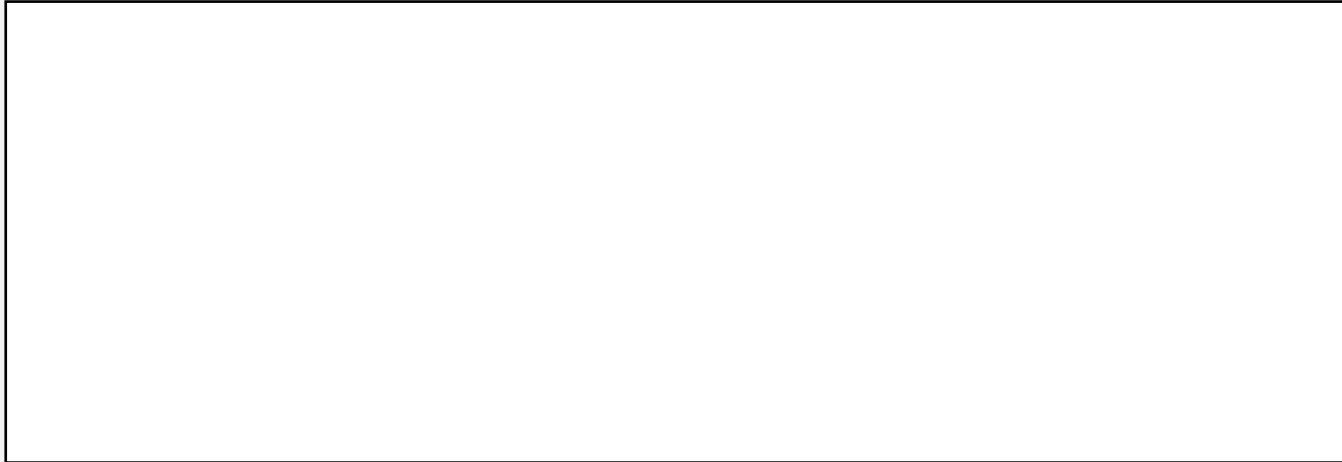
```
HELLO
```

```
DON'T GIVE UP
```

```
YOU'RE DOING GREAT
```



■ 과제 2



■ 과제 3

- 터미널에서 원본 파일명(halo.txt)과 복사될 파일명(halo_cp.txt)을 입력받아 내용을 복사하는 프로그램(test3.py)을 작성하시오.
- 조건
 - 명령 인자 사용 : sys.argv 를 사용하여 인자를 입력받기
 - 파일 읽고 쓰기 : 원본 파일은 읽기 모드(r), 사본 파일은 쓰기 모드(w)로 열기
 - 파일 인코딩 형식 'utf-8' 사용할 것.

```
# halo.txt
```

```
hello  
don't give up  
you're doing great
```

실행 예시

>>> python test3.py halo.txt halo_cp.txt
파일 복사가 완료되었습니다.

원본 파일명

복사될 파일명

■ 과제 3

```
#test3.py
```

```
import 
```

```
src =   
dst = 
```

```
(src, , encoding=) as f_src:  
     = 
```

```
(dst, , encoding=) as f_dst:  
    
```

```
print("파일 복사가 완료되었습니다.")
```

■ 과제 4

- 여러 개의 숫자를 입력받음 = 입력받는 숫자의 개수는 고정하지 않음.
- 입력받은 숫자의 제곱 값을 계산함.
- 계산된 결과를 리스트 형태로 반환하는 `get_even_squares()` 함수를 작성하시오

```
def get_even_squares():
```

```
    return result
```

```
print(get_even_squares(1, 2, 3, 4, 5, 6))  
print(get_even_squares(10, 20, 30))
```

✓ 0.0s

```
[4, 16, 36]  
[100, 400, 900]
```